

计算机与程序设计基础 (C)

实验课堂

第四讲 数组与指针 (2)



任务一 再谈野指针

- 释放了内存，却继续使用它
 - 千万不要以为指针所指的内存被释放以后，指针就会变成NULL或者消亡了，其实它的地址并没有改变，仍然指向这块内存
 - 释放内存的结果只是改变了内存内容，使其变成了垃圾内存
- 指向垃圾内存的指针被称为“野指针”
- 用判断指针是否为NULL不能避免使用野指针的问题，除非在释放指针所指向的内存后，立即人为地在程序中将该指针置为NULL

任务一 再谈野指针

- 【错误案例】从键盘任意输入一串字符，然后在屏幕上输出。
- **warning**: function returns address of local variable [-Wreturn-local-addr]
- 野指针错误通常都发生在试图从一个函数返回指向局部变量的地址，因为系统给函数中声明的局部变量分配的内存，在函数调用结束后就被自动释放了。

```
1  #include <stdio.h>
2  char * GetStr(void);
3  int main(void)
4  {
5      char *ptr=NULL;
6      ptr=GetStr();
7      puts(ptr);
8      return 0;
9  }
10
11 char* GetStr(void)
12 {
13     char s[80];
14     scanf("%s", s);
15     return s;
16 }
```

asdf

Process returned 0 (0x0) execution time : 4.982 s
Press any key to continue.

任务一 再谈野指针

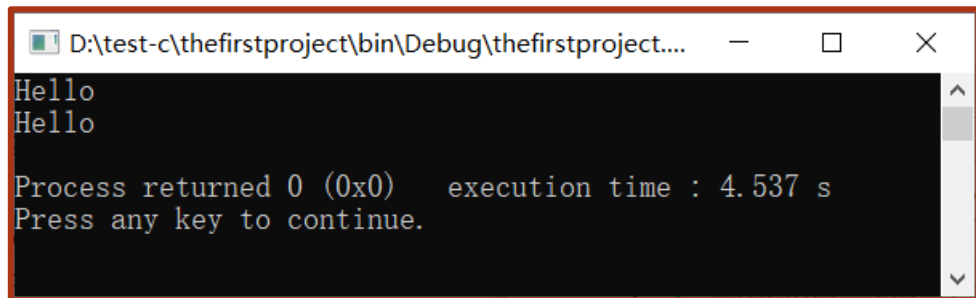
```
1  #include <stdio.h>
2  void GetStr(char *);
3  int main(void)
4  {
5      char s[80];
6      char *ptr=s;
7      GetStr(ptr);
8      puts(ptr);
9      return 0;
10 }
11
12 void GetStr(char *s)
13 {
14     scanf("%s", s);
15 }
```

```
sdfdf
sdfdf
```

```
Process returned 0 (0x0)   execution time : 3.400 s
Press any key to continue.
```

任务一 再谈野指针

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  char* GetStr(char *);
4  int main(void)
5  {
6      char *ptr=NULL;
7      ptr=GetStr(ptr);
8      puts(ptr);
9      free(ptr);
10     return 0;
11 }
12
13 char* GetStr(char *s)
14 {
15     s=(char*) malloc(100);
16     scanf("%s", s);
17     return s;
18 }
```



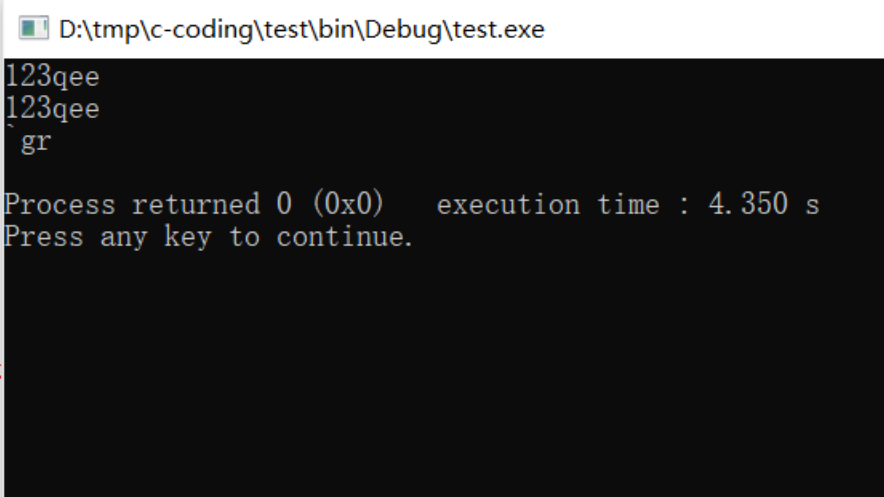
```
D:\test-c\thefirstproject\bin\Debug\thefirstproject....
Hello
Hello

Process returned 0 (0x0)   execution time : 4.537 s
Press any key to continue.
```

任务一 再谈野指针

- 调用free释放内存之后

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  char* GetStr(char *);
4  int main(void)
5  {
6      char *ptr=NULL;
7      ptr=GetStr(ptr);
8      puts(ptr);
9      free(ptr);
10     puts(ptr);
11     return 0;
12 }
13
14 char* GetStr(char *s)
15 {
16     s=(char*) malloc(100);
17     scanf("%s", s);
18     return s;
19 }
20
```



```
D:\tmp\c-coding\test\bin\Debug\test.exe
123qee
123qee
gr
Process returned 0 (0x0) execution time : 4.350 s
Press any key to continue.
```

任务二 打印二维数组的地址和元素值

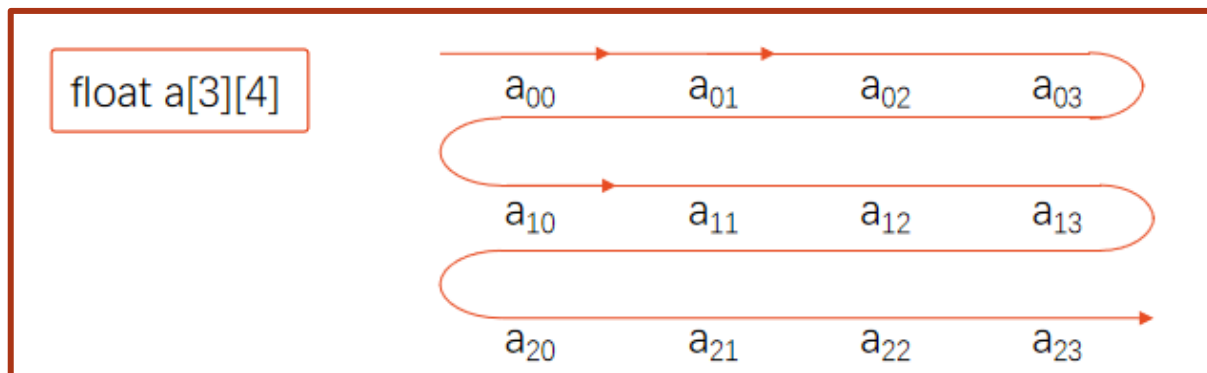
```
1  #include <stdio.h>
2  int main()
3  { int a[3][4]={1,3,5,7,9,11,13,15,17,19,21,23};
4      printf("%d,%d\n",a,*a);           // 0行首地址和0行0列元素地址
5      printf("%d,%d\n",a[0],*(a+0));    // 0行0列元素地址
6      printf("%d,%d\n",&a[0],&a[0][0]);  // 0行首地址和0行0列元素地址
7      printf("%d,%d\n",a[1],a+1);       // 1行0列元素地址和1行首地址
8      printf("%d,%d\n",&a[1][0],*(a+1)+0); // 1行0列元素地址
9      printf("%d,%d\n",a[2],*(a+2));    // 2行0列元素地址
10     printf("%d,%d\n",&a[2],a+2);      // 2行首地址
11     printf("%d,%d\n",a[1][0],*(*(a+1)+0)); // 1行0列元素的值
12     printf("%d,%d\n",*a[2],*(*(a+2)+0)); // 2行0列元素的值
13     return 0;
14 }
```

```
6422000, 6422000
6422000, 6422000
6422000, 6422000
6422016, 6422016
6422016, 6422016
6422032, 6422032
6422032, 6422032
9, 9
17, 17
```

任务三 用指针变量访问二维数组——指向元素

```
1  #include <stdio.h>
2  int main()
3  {int a[3][4]={1,3,5,7,9,11,13,15,17,19,21,23};
4    int *p;
5    for(p=a[0];p<a[0]+12;p++)
6        {if((p-a[0])%4==0)printf("\n");
7          printf("%4d",*p);
8        }
9    printf("\n");
10   return 0;
11 }
```

1	3	5	7
9	11	13	15
17	19	21	23



2000	a[0][0]	第0行元素
2004	a[0][1]	
2008	a[0][2]	
2012	a[0][3]	
2016	a[1][0]	第1行元素
2020	a[1][1]	
2024	a[1][2]	
2028	a[1][3]	
2032	a[2][0]	第2行元素
2036	a[2][1]	
2040	a[2][2]	
2044	a[2][3]	

任务四 用指针变量访问二维数组——指向行

```
1  #include <stdio.h>
2  int main()
3  {int a[3][4]={1,3,5,7,9,11,13,15,17,19,21,23};
4    int (*p)[4],i,j;           // 指针变量p指向包含4个整型元素的一维数组
5    p=a;                       // p指向二维数组的0行
6    printf("please enter row and colum:");
7    scanf("%d,%d",&i,&j);       // 指定元素的行列
8    printf("a[%d,%d]=%d\n",i,j,*(*(p+i)+j)); // 输出a[i][j]的值
9    return 0;
10 }
```

```
please enter row and colum:2,3
a[2,3]=23
```

	队员1	队员2	队员3	队员4
第1分队	2456	1847	1243	1600
第2分队	3045	2018	1725	2020
第3分队	1427	1175	1046	1976

a[0] —— a[0][0] a[0][1] a[0][2] a[0][3]

a[1] —— a[1][0] a[1][1] a[1][2] a[1][3]

a[2] —— a[2][0] a[2][1] a[2][2] a[2][3]

任务五 围圈报数(使用指针)

- N个人围成一圈，顺序排号。从第1个人开始报数（从1到3报数），凡报到3的人退出圈子，问最后留下的是原来第几号的人。



- 怎么表示序号?
- 怎么表示退出?
- 怎么记录1-2-3报数?
- 怎么实现围“圈”?
- 怎么判断循环停止?
- ...

```
for(i=0; i<n; i++)  
    *(p+i)=i+1; //以1至n为序给每个人编号
```